

Bazy Danych - Zadanie 3

1 DDL (Data Definition Language)

Zarządzanie strukturami bazy danych.

- **CREATE TABLE:** Rozszerzona składnia z różnymi więzami integralności (Constraints).

```
CREATE TABLE Klienci (  
    KlientID INT IDENTITY(1,1) NOT NULL, -- Automatyczna inkrementacja  
    Nazwa VARCHAR(100) NOT NULL,  
    Pesel CHAR(11) UNIQUE, -- Wymusza unikalność  
    Wiek INT CHECK (Wiek >= 18), -- Sprawdza warunek logiczny  
    DataRejestracji DATETIME DEFAULT GETDATE(), -- Wartość domyślna  
    KrajID INT NULL,  
    CONSTRAINT PK_Klienci PRIMARY KEY (KlientID),  
    CONSTRAINT FK_Klienci_Kraje FOREIGN KEY (KrajID) REFERENCES Kraje(KrajID)  
);
```

- **ALTER TABLE:** Dodawanie/usuwanie kolumn i więzów.

```
ALTER TABLE Klienci ADD Email VARCHAR(255) NULL;  
ALTER TABLE Klienci ALTER COLUMN Nazwa VARCHAR(150) NOT NULL;  
ALTER TABLE Klienci DROP COLUMN Wiek;  
ALTER TABLE Klienci ADD CONSTRAINT FK_Noway FOREIGN KEY (Kolumna) REFERENCES Inna(Id);
```

- **DROP:** Usuwanie struktur.

```
DROP TABLE Klienci; -- Usuwa tabele i jej dane
```

2 DML (Data Manipulation Language)

Operacje na danych zawartych w tabelach.

- **INSERT:**

```
-- Podstawowe wstawianie  
INSERT INTO Kraje (Nazwa, Kod) VALUES ('Polska', 'PL');  
  
-- Wstawianie wielu wierszy naraz  
INSERT INTO Kraje (Nazwa, Kod) VALUES ('Niemcy', 'DE'), ('Francja', 'FR');  
  
-- Wstawianie danych z innej tabeli (np. archiwizacja)  
INSERT INTO ArchiwumZamowien (ZamowienieID, Data)  
SELECT ZamowienieID, DataZamowienia FROM Zamowienia WHERE YEAR(DataZamowienia) < 2000;
```

- **UPDATE:** Modyfikacja danych. Bardzo przydatna jest składnia z użyciem JOIN (specyficzna dla T-SQL).

```
-- Klasyczny UPDATE  
UPDATE Produkty SET Cena = Cena * 1.1 WHERE Kategoria = 'Elektronika';  
  
-- UPDATE wykorzystujący JOIN (np. obniżka dla stałych klientów)  
UPDATE z  
SET z.Kwota = z.Kwota * 0.9  
FROM Zamowienia z  
JOIN Klienci k ON z.KlientID = k.KlientID  
WHERE k.Status = 'VIP';
```

- **DELETE:** Usuwanie danych z uwzględnieniem podzapytań.

```
DELETE FROM Zamowienia  
WHERE KlientID IN (SELECT KlientID FROM Klienci WHERE Aktywny = 0);
```

3 Indeksy (Wydażność)

- **Heap (Sterta):** Tabela bez indeksu zgrupowanego. Dane leżą chaotycznie.
- **Clustered Index:** Sortuje fizycznie dane w tabeli. Może być tylko jeden.
- **Non-clustered Index:** "Spis treści" ze wskaźnikami do fizycznych wierszy.

```
-- Zwykly indeks niezgrupowany (przyspiesza wyszukiwanie np. po nazwisku)
CREATE NONCLUSTERED INDEX IX_Pracownicy_Nazwisko ON Pracownicy (Nazwisko);

-- Indeks wielokolumnowy (kolejnosc kolumn ma znaczenie!)
CREATE INDEX IX_Zamowienia_Data_Klient ON Zamowienia (DataZamowienia DESC, KlientID ASC);

-- Indeks unikalny (nie pozwala na duplikaty w tych dwoch kolumnach)
CREATE UNIQUE NONCLUSTERED INDEX UX_KodyRabatowe ON Rabaty (Kod);

-- Usuwanie indeksu
DROP INDEX IX_Pracownicy_Nazwisko ON Pracownicy;
```

4 Transakcje (ACID) i Poziomy Izolacji

Transakcja to zbiór operacji traktowanych jako niepodzielna całość. Spełnia właściwości ACID (Atomicity, Consistency, Isolation, Durability).

Aby zrozumieć poziomy izolacji, najpierw należy poznać **trzy anomalie** (zjawiska), które mogą wystąpić w środowisku współbieżnym (gdy wielu użytkowników modyfikuje dane jednocześnie):

- **Dirty Read (Brudny odczyt):** Odczytanie danych, które zostały zmodyfikowane przez inną transakcję, ale nie zostały jeszcze przez nią zatwierdzone (zrobiono UPDATE, ale nie zrobiono COMMIT). Jeśli tamta transakcja zrobi ROLLBACK, zostajemy z nieistniejącymi danymi.
- **Non-repeatable Read (Niepowtarzalny odczyt):** Transakcja odczytuje ten sam wiersz dwa razy. W międzyczasie inna transakcja modyfikuje ten wiersz i go zatwierdza. Drugi odczyt daje inne wyniki niż pierwszy.
- **Phantom Read (Odczyt widmo):** Transakcja wyszukuje wiersze spełniające dany warunek (np. Wiek > 18). W międzyczasie inna transakcja wstawia nowe rekordy, które spełniają ten warunek. Przy ponownym wyszukiwaniu w tej samej transakcji pojawiają się "wiersze widma", których wcześniej nie było.

Poziomy Izolacji (Isolation Levels)

Im wyższy poziom izolacji, tym większa spójność danych, ale mniejsza wydajność i większe ryzyko blokowania się zapytań (deadlocks).

1. **READ UNCOMMITTED:** Najniższy poziom. Pozwala na wszystko, w tym na *Dirty Read*. Baza nie zakłada blokad na odczytywane dane, przez co jest ekstremalnie szybka, ale ryzykowna (np. wyliczanie statystyk, w których dokładność co do grosza nie jest wymagana).
2. **READ COMMITTED:** Domyślny poziom w SQL Server. Zapobiega *Dirty Read* – transakcja czyta tylko zatwierdzone dane. Nadal mogą jednak wystąpić *Non-repeatable Read* oraz *Phantom Read*.
3. **REPEATABLE READ:** Zapobiega *Dirty Read* i *Non-repeatable Read*. Jeśli transakcja odczytała wiersz, zakłada na nim blokadę współdzieloną – inna transakcja nie może go zmodyfikować ani usunąć, dopóki nasza się nie zakończy. Nadal mogą się pojawiać *Phantom Read* (można dodawać nowe wiersze do tabeli).
4. **SERIALIZABLE:** Najwyższy standardowy poziom izolacji. Transakcje zachowują się tak, jakby były wykonywane w kolejności, jedna po drugiej. Gwarantuje absolutny brak *Dirty*, *Non-repeatable* i *Phantom Reads*. Baza zakłada blokady na całe zakresy danych (Range Locks). Bardzo mocno obciąża system.
5. **SNAPSHOT** (Dodatkowy w SQL Server): Używa wersjonowania wierszy (zapisuje stare wersje w systemowej bazie TempDB) zamiast agresywnego blokowania. Zapobiega wszystkim anomalom, ale czyta spójną "migawkę" danych z momentu startu transakcji.

Składnia z użyciem TRY...CATCH:

```
-- Ustawienie poziomu izolacji (deklarujemy przed uruchomieniem transakcji)
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION;
BEGIN TRY
```

```

-- Przelewy między kontami (muszą być wykonane razem - Atomicity)
UPDATE Konta SET Saldo = Saldo - 100 WHERE KontoID = 1;
UPDATE Konta SET Saldo = Saldo + 100 WHERE KontoID = 2;

-- Zatwierdzenie (Commit) - utrwalenie zmian na dysku (Durability)
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Wycofanie (Rollback) - przywracanie stanu początkowego w razie błędu
    ROLLBACK TRANSACTION;

    -- Zwrocenie informacji o błędzie
    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    RAISERROR (@ErrorMessage, 16, 1);
END CATCH

```

5 Procedury Składowane (Stored Procedures)

Zaawansowana składnia z parametrami wejściowymi (IN) i wyjściowymi (OUTPUT).

```

CREATE PROCEDURE DodajZamowienie
    @KlientID INT,
    @Kwota MONEY,
    @NoweZamowienieID INT OUTPUT -- Parametr zwracający wartość
AS
BEGIN
    SET NOCOUNT ON; -- Zwiększa wydajność (nie wysyła komunikatów o liczbie wierszy)

    -- Prosta logika warunkowa IF/ELSE
    IF NOT EXISTS (SELECT 1 FROM Klienci WHERE KlientID = @KlientID)
    BEGIN
        PRINT 'Klient nie istnieje!';
        RETURN -1; -- Przerywa wykonanie procedury
    END

    INSERT INTO Zamowienia (KlientID, Kwota, Data) VALUES (@KlientID, @Kwota, GETDATE());

    -- Złapanie nowo wygenerowanego ID i przypisanie do parametru wyjściowego
    SET @NoweZamowienieID = SCOPE_IDENTITY();
END;
GO

-- WYWOŁANIE PROCEDURY Z PARAMETREM OUTPUT
DECLARE @WygenerowaneID INT;
EXECUTE DodajZamowienie
    @KlientID = 5,
    @Kwota = 250.00,
    @NoweZamowienieID = @WygenerowaneID OUTPUT;

SELECT @WygenerowaneID AS 'Nowe ID';

```

6 Wyzwalacze (Triggers)

Automatyczne akcje bazujące na tabelach wirtualnych inserted (nowe dane) i deleted (stare dane).

```

-- Trigger logujący zmiany w pensjach
CREATE TRIGGER trg_LogujZmianyPensji ON Pracownicy
AFTER UPDATE
AS
BEGIN
    -- Funkcja UPDATE(kolumna) sprawdza, czy konkretna kolumna była modyfikowana
    IF UPDATE(Pensja)
    BEGIN
        INSERT INTO HistoriaPensji (PracownikID, StaraPensja, NowaPensja, DataZmiany)
        SELECT
            i.PracownikID,
            d.Pensja AS StaraPensja,
            i.Pensja AS NowaPensja,
            GETDATE()
        FROM inserted i
        JOIN deleted d ON i.PracownikID = d.PracownikID
        WHERE i.Pensja <> d.Pensja; -- Rejestruj tylko jeśli fizycznie uległa zmianie
    END
END;

```

7 CTE (Common Table Expressions) i Funkcje Okna

Wielokrotne CTE w jednym zapytaniu oraz partycjonowanie.

```
-- Deklaracja wielu CTE po przecinku
WITH ZamowieniaWzbogacone AS (
    SELECT z.ZamowienieID, z.KlientID, z.Kwota,
           -- ROW_NUMBER numeruje zamowienia klienta chronologicznie
           ROW_NUMBER() OVER(PARTITION BY z.KlientID ORDER BY z.Data ASC) as NumerZamowieniaKlienta,
           -- SUM OVER liczy sume krocza (lub calkowita) dla klienta
           SUM(z.Kwota) OVER(PARTITION BY z.KlientID) as Laczniewydal
    FROM Zamowienia z
),
KlienciPremium AS (
    SELECT KlientID FROM ZamowieniaWzbogacone WHERE Laczniewydal > 10000
)
-- Koncowe zapytanie laczące dane z dwóch powyższych CTE
SELECT zw.* FROM ZamowieniaWzbogacone zw
JOIN KlienciPremium kp ON zw.KlientID = kp.KlientID
WHERE zw.NumerZamowieniaKlienta = 1; -- Znajdz pierwsze zamowienie klientow VIP
```

8 Widoki (Views)

Widok to wirtualna tabela, której zawartość jest definiowana przez zapytanie SELECT. Widoki nie przechowują danych (z wyjątkiem widoków indeksowanych), a jedynie definicję zapytania, które jest wykonywane każdorazowo przy odwołaniu do widoku.

- Zalety:

- Uproszczenie złożonych zapytań (hermetyzacja logiki).
- Bezpieczeństwo: ograniczanie dostępu do konkretnych kolumn (np. ukrywanie danych płacowych).
- Zapewnienie spójnego interfejsu dla aplikacji (jeśli zmieni się struktura tabel, wystarczy zaktualizować widok).

Składnia tworzenia i zarządzania

```
-- 1. Tworzenie widoku
CREATE VIEW v_AktywniKlienci AS
SELECT KlientID, Nazwa, Email
FROM Klienci
WHERE Aktywny = 1;

-- 2. Korzystanie z widoku (jak ze zwykłej tabeli)
SELECT * FROM v_AktywniKlienci WHERE Nazwa LIKE 'A%';

-- 3. Modyfikacja widoku (bez jego usuwania)
ALTER VIEW v_AktywniKlienci AS
SELECT KlientID, Nazwa, Email, Telefon
FROM Klienci
WHERE Aktywny = 1;

-- 4. Usuwanie widoku
DROP VIEW v_AktywniKlienci;
```

Opcje zaawansowane

```
-- WITH SCHEMABINDING:
-- Zapobiega usunięciu lub zmianie kolumn w tabelach bazowych,
-- na których opiera się widok. Wymaga używania schematu (np. dbo.).
CREATE VIEW v_ZamowieniaSzczegoly WITH SCHEMABINDING AS
SELECT z.ZamowienieID, s.ProduktID, s.Ilosc
FROM dbo.Zamowienia z
JOIN dbo.SzczegolyZamowien s ON z.ZamowienieID = s.ZamowienieID;

-- WIDOKI ZASZYFROWANE (Encryption):
-- Kod źródłowy widoku nie jest widoczny w systemowych tabelach baz danych.
CREATE VIEW v_TajneDane WITH ENCRYPTION AS
SELECT * FROM DaneFirma;
```

Widok vs CTE - kluczowe różnice

- **Czas życia:** Widok jest trwałym obiektem w bazie danych. CTE jest tymczasowe i istnieje tylko podczas wykonania zapytania.
- **Reużywalność:** Widok można wywołać w wielu różnych zapytaniach. CTE jest ograniczone do jednego zapytania, bezpośrednio po którym jest zdefiniowane.
- **Rekurencja:** CTE wspiera mechanizm rekurencji (odwoływanie się do samego siebie), co jest niezbędne przy przetwarzaniu struktur hierarchicznych (np. drzewa kategorii produktów czy struktury podległości pracowników).

9 Kursory - Szczegółowe rozwinięcie składni

Kursor w SQL Server to mechanizm umożliwiający przetwarzanie wyników zapytania wiersz po wierszu. Jego pełna deklaracja wygląda następująco:

```
DECLARE kursor_nazwa CURSOR [LOCAL | GLOBAL] [FORWARD_ONLY | SCROLL]
[STATIC | KEYSET | DYNAMIC | FAST_FORWARD]
FOR SELECT_STATEMENT;
```

Wyjaśnienie parametrów deklaracji:

- **LOCAL vs GLOBAL:**
 - **LOCAL:** Kursor jest dostępny tylko wewnątrz partii (batch), procedury lub wyzwalacza, w którym został zadeklarowany. Jest to zalecane ustawienie (bezpieczniejsze).
 - **GLOBAL:** Kursor jest widoczny w całym połączeniu z bazą danych (sesji).
- **FORWARD_ONLY vs SCROLL:**
 - **FORWARD_ONLY:** (Domyślne) Kursor można przesuwać tylko w jednym kierunku: od pierwszego do ostatniego wiersza (FETCH NEXT). Jest to najszybszy typ kursora.
 - **SCROLL:** Pozwala na dowolne poruszanie się po kursorze: FETCH NEXT, FETCH PRIOR (wstecz), FETCH FIRST, FETCH LAST oraz FETCH ABSOLUTE n.
- **Typy kursorów (Wydajność):**
 - **STATIC:** Tworzy tymczasową kopię danych w bazie tempdb. Zmiany w oryginalnych tabelach nie są widoczne w kursorze.
 - **KEYSET:** Zapamiętuje klucze wierszy w tempdb. Zmiany wartości wierszy są widoczne, ale wstawianie nowych wierszy przez inne transakcje nie.
 - **DYNAMIC:** Najbardziej elastyczny; odzwierciedla wszystkie zmiany w danych w czasie rzeczywistym. Najwolniejszy.
 - **FAST_FORWARD:** To optymalizacja dla kursorów typu FORWARD_ONLY. SQL Server automatycznie optymalizuje plan wykonania, aby był tak szybki, jak to tylko możliwe. To najczęściej wybierany typ kursora, jeśli nie potrzebujesz przewijać danych wstecz.

Instrukcje operacyjne

- **OPEN:** Alokuję zasoby i wykonuje zapytanie SELECT.
- **FETCH NEXT:** Pobiera kolejny wiersz do zadeklarowanych zmiennych.
- **@@FETCH_STATUS:** Funkcja systemowa zwracająca stan ostatniej operacji FETCH:
 - 0: Sukces.
 - -1: Koniec kursora (nie udało się pobrać wiersza).
 - -2: Błąd (wiersz został usunięty w międzyczasie).
- **CLOSE:** Zwalnia blokady (jeśli były założone) i pozwala na ponowne otwarcie (OPEN) bez ponownej deklaracji.
- **DEALLOCATE:** Całkowicie usuwa kursor z pamięci serwera (zwalnia nazwę).

10 Lab06 Scenariusz 1

```
-- 1. Tworzenie tabeli ArchivedOrders
CREATE TABLE ArchivedOrders (
    OrderID INT NOT NULL,
    CustomerID NCHAR(5) NULL,
    EmployeeID INT NULL,
    OrderDate DATETIME NULL,
    RequiredDate DATETIME NULL,
    ShippedDate DATETIME NULL,
    ShipVia INT NULL,
    Freight MONEY NULL,
    ShipName NVARCHAR(40) NULL,
    ShipAddress NVARCHAR(60) NULL,
    ShipCity NVARCHAR(15) NULL,
    ShipRegion NVARCHAR(15) NULL,
    ShipPostalCode NVARCHAR(10) NULL,
    ShipCountry NVARCHAR(15) NULL,
    ArchiveDate DATETIME NOT NULL,
    CONSTRAINT PK_ArchivedOrders PRIMARY KEY (OrderID),
    CONSTRAINT FK_ArchivedOrders_Customers FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
    CONSTRAINT FK_ArchivedOrders_Employees FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID)
);

-- 2. Tworzenie tabeli ArchivedOrderDetails
CREATE TABLE ArchivedOrderDetails (
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    UnitPrice MONEY NOT NULL,
    Quantity SMALLINT NOT NULL,
    Discount REAL NOT NULL,
    CONSTRAINT PK_ArchivedOrderDetails PRIMARY KEY (OrderID, ProductID),
    CONSTRAINT FK_ArchivedOrderDetails_ArchivedOrders FOREIGN KEY (OrderID) REFERENCES ArchivedOrders(OrderID),
    CONSTRAINT FK_ArchivedOrderDetails_Products FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

-- 3. Przenoszenie danych (Atomowosc transakcji)
BEGIN TRANSACTION;
-- Kopiowanie naglowkow z 1996 roku
INSERT INTO ArchivedOrders (...)
SELECT ..., GETDATE() FROM Orders WHERE YEAR(OrderDate) = 1996;

-- Kopiowanie detali
INSERT INTO ArchivedOrderDetails (...)
SELECT od.OrderID, od.ProductID, od.UnitPrice, od.Quantity, od.Discount
FROM [Order Details] od JOIN Orders o ON od.OrderID = o.OrderID
WHERE YEAR(o.OrderDate) = 1996;

-- Usuwanie z oryginalnych tabel (najpierw szczegoly, potem zamowienie)
DELETE FROM [Order Details] WHERE OrderID IN (SELECT OrderID FROM Orders WHERE YEAR(OrderDate) = 1996);
DELETE FROM Orders WHERE YEAR(OrderDate) = 1996;
COMMIT TRANSACTION;
```

11 Lab06 Scenariusz 2

```
-- Dodanie kolumny IsCancelled do tabeli Orders
ALTER TABLE Orders
    ADD IsCancelled INT;

-- Transakcja wprowadzajaca zmiany
BEGIN TRANSACTION;

-- 1. Ustawienie IsCancelled = 0 dla wszystkich klientow oprócz ALFKI
UPDATE Orders
SET IsCancelled = 0
WHERE CustomerID <> 'ALFKI';

-- 2. Ustawienie IsCancelled = 1 dla zamowien klienta ALFKI
UPDATE Orders
SET IsCancelled = 1
WHERE CustomerID = 'ALFKI';
```

```

-- 3. Wyzerowanie ilosci (Quantity) w szczegolach zamowien dla klienta ALFKI.
-- Tabela [Order Details] nie ma kolumny CustomerID, wiec musimy
-- znalezc odpowiednie OrderID w tabeli Orders za pomoca podzapytania.
UPDATE [Order Details]
SET Quantity = 0
WHERE OrderID IN (
    SELECT OrderID
    FROM Orders
    WHERE CustomerID = 'ALFKI'

-- Alternatywna metoda: UPDATE z uzyciem JOIN
-- UPDATE od
-- SET od.Quantity = 0
-- FROM [Order Details] od
-- JOIN Orders o ON od.OrderID = o.OrderID
-- WHERE o.CustomerID = 'ALFKI';
);

COMMIT TRANSACTION;

```

12 Lab06 Scenariusz 3

```

-- 1. Utworzenie tabeli PriceList
CREATE TABLE PriceList (
    productId INT NOT NULL,
    price MONEY NOT NULL,
    date_from DATETIME NOT NULL,
    date_to DATETIME NULL,

    -- Klucz podstawowy
    CONSTRAINT PK_PriceList PRIMARY KEY (productId, date_from),

    -- Klucz obcy do tabeli Products
    CONSTRAINT FK_PriceList_Products FOREIGN KEY (productId) REFERENCES Products(
        ProductID)
);

-- 2. Wstawienie testowych cen dla poszczegolnych lat
INSERT INTO PriceList (productId, price, date_from, date_to)
SELECT
    p.ProductID,
    -- Symulacja zmiany ceny: cena bazowa * mnoznik zalezny od roku
    p.UnitPrice * (1.0 + (Lata.OrderYear - 1996) * 0.1) AS test_price,
    -- Ustawienie poczatku roku (1 stycznia)
    DATETIMEFROMPARTS(Lata.OrderYear, 1, 1, 0, 0, 0, 0) AS date_from,
    -- Ustawienie konca roku (31 grudnia)
    DATETIMEFROMPARTS(Lata.OrderYear, 12, 31, 23, 59, 59, 999) AS date_to
FROM Products p
CROSS JOIN (
    -- Wyciagamy unikalne lata z tabeli Orders
    SELECT DISTINCT YEAR(OrderDate) AS OrderYear
    FROM Orders
    WHERE OrderDate IS NOT NULL
) Lata;

-- 3. Dodanie kolumny TotalValue do tabeli Orders
ALTER TABLE Orders
ADD TotalValue MONEY;

-- 4. Aktualizacja TotalValue na podstawie cennika obowiazujacego w dniu zamowienia
UPDATE o
SET TotalValue = (
    -- Podzapytanie liczace calkowita wartosc zamowienia
    SELECT SUM(od.Quantity * pl.price)
    FROM [Order Details] od
        JOIN PriceList pl ON od.ProductID = pl.productId
    -- Szukamy ceny obowiazujacej w momencie zlozenia zamowienia o.OrderDate
    WHERE od.OrderID = o.OrderID
        AND o.OrderDate >= pl.date_from
        AND o.OrderDate <= pl.date_to
)
FROM Orders o;

```

13 Lab10 Scenariusz 1

```
DROP PROCEDURE IF EXISTS ArchiveOrders;
GO

CREATE PROCEDURE ArchiveOrders
@MinimalAgeInYears INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Deklaracja zmiennej lokalnej i zapisanie do niej bieżącej daty i czasu
    DECLARE @CurrentDate DATETIME;
    SET @CurrentDate = GETDATE();

    BEGIN TRANSACTION;

    BEGIN TRY
        -- Krok 1: Wstawiamy nagłówki zamówień
        INSERT INTO ArchivedOrders (
            OrderID, CustomerID, EmployeeID, OrderDate, RequiredDate, ShippedDate,
            ShipVia, Freight, ShipName, ShipAddress, ShipCity, ShipRegion,
            ShipPostalCode, ShipCountry, ArchiveDate
        )
        SELECT
            OrderID, CustomerID, EmployeeID, OrderDate, RequiredDate, ShippedDate,
            ShipVia, Freight, ShipName, ShipAddress, ShipCity, ShipRegion,
            ShipPostalCode, ShipCountry,
            @CurrentDate
        FROM Orders
        WHERE DATEDIFF(year, OrderDate, @CurrentDate) >= @MinimalAgeInYears;

        -- Krok 2: Wstawiamy szczegóły zamówień
        INSERT INTO ArchivedOrderDetails (OrderID, ProductID, UnitPrice, Quantity, Discount)
        SELECT od.OrderID, od.ProductID, od.UnitPrice, od.Quantity, od.Discount
        FROM [Order Details] od
        JOIN Orders o ON od.OrderID = o.OrderID
        WHERE DATEDIFF(year, o.OrderDate, @CurrentDate) >= @MinimalAgeInYears;

        -- Krok 3: Usuwanie przeniesionych detali z oryginalnej tabeli
        DELETE FROM [Order Details]
        WHERE OrderID IN (
            SELECT OrderID
            FROM Orders
            WHERE DATEDIFF(year, OrderDate, @CurrentDate) >= @MinimalAgeInYears
        );

        -- Krok 4: Usuwanie przeniesionych nagłówków z oryginalnej tabeli
        DELETE FROM Orders
        WHERE DATEDIFF(year, OrderDate, @CurrentDate) >= @MinimalAgeInYears;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

14 Lab10 Scenariusz 2 - CTE

```
DROP PROCEDURE IF EXISTS CalculateCustomerDiscounts;
GO

CREATE PROCEDURE CalculateCustomerDiscounts
@CustomerId NCHAR(5) -- Parametr określający klienta
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRANSACTION;

    BEGIN TRY
        -- Używamy CTE, aby "ponumerować" zamówienia
        -- danego produktu dla podanego klienta w czasie.
```



```

WITH RankedProductOrders AS (
    SELECT
        od.OrderID,
        od.ProductID,
        -- ROW_NUMBER() numeruje wiersze od 1.
        -- Odejmujac 1, otrzymujemy dokładnie "liczbe wcześniejszych zamówien" (0, 1, 2, 3...)
        ROW_NUMBER() OVER(
            PARTITION BY od.ProductID -- Numerujemy osobno dla kazdego produktu
            ORDER BY o.OrderDate ASC, o.OrderID ASC -- Chronologicznie
        ) - 1 AS PreviousOrdersCount
    FROM [Order Details] od
        JOIN Orders o ON od.OrderID = o.OrderID
    WHERE o.CustomerID = @CustomerId
)
-- Glowna aktualizacja korzystajaca z naszych wyliczen w CTE
UPDATE od
SET Discount = CASE
    WHEN r.PreviousOrdersCount = 0 THEN 0.00 -- Pierwsze
        zamowienie: zniżka 0%
    WHEN r.PreviousOrdersCount IN (1, 2) THEN 0.05 -- 1 do 2
        wcześniejszych: zniżka 5%
    WHEN r.PreviousOrdersCount = 3 THEN 0.10 -- 3 wcześniejsze:
        zniżka 10%
    WHEN r.PreviousOrdersCount > 3 THEN 0.20 -- Powyżej 3: zniżka
        20%
END
FROM [Order Details] od
    JOIN RankedProductOrders r ON od.OrderID = r.OrderID AND od.ProductID = r.ProductID
;

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

```

15 Lab10 Scenariusz 2 - Cursor

```

DROP PROCEDURE IF EXISTS CalculateCustomerDiscounts_Cursor;
GO

CREATE PROCEDURE CalculateCustomerDiscounts_Cursor
@CustomerId NCHAR(5)
AS
BEGIN
    SET NOCOUNT ON;

    -- Deklaracja zmiennych, do których będziemy wczytywać dane wiersz po wierszu z kursora
    DECLARE @CurrentOrderID INT;
    DECLARE @CurrentProductID INT;
    DECLARE @CurrentOrderDate DATETIME;

    -- Zmienne pomocnicze do obliczen wewnątrz petli
    DECLARE @PreviousOrdersCount INT;
    DECLARE @NewDiscount REAL;

    BEGIN TRANSACTION;

    BEGIN TRY
        -- 1. DECLARE: Definiujemy kursor. Wybieramy zamówienia klienta, chronologicznie.
        DECLARE order_cursor CURSOR FOR
            SELECT od.OrderID, od.ProductID, o.OrderDate
            FROM [Order Details] od
                JOIN Orders o ON od.OrderID = o.OrderID
            WHERE o.CustomerID = @CustomerId
            ORDER BY o.OrderDate ASC, od.OrderID ASC;

        -- 2. OPEN: Otwieramy kursor
        OPEN order_cursor;

        -- 3. FETCH: Pobieramy pierwszy wiersz do naszych zmiennych
        FETCH NEXT FROM order_cursor INTO @CurrentOrderID, @CurrentProductID, @CurrentOrderDate;
    
```

```

-- 4. PETLA: @@FETCH_STATUS = 0 oznacza, ze pomyslnie pobrano wiersz
WHILE @@FETCH_STATUS = 0
BEGIN

    -- KROK A: Liczymy ile bylo zamowien na ten sam produkt wzczesniej (historycznie)
    SELECT @PreviousOrdersCount = COUNT(*)
    FROM [Order Details] od2
        JOIN Orders o2 ON od2.OrderID = o2.OrderID
    WHERE o2.CustomerID = @CustomerId
        AND od2.ProductID = @CurrentProductID
    -- Musi byc to zamowienie wzczesniejsze (starsza data, lub ta sama data ale
    -- mniejsze ID)
    AND (o2.OrderDate < @CurrentOrderDate
        OR (o2.OrderDate = @CurrentOrderDate AND o2.OrderID < @CurrentOrderID));

    -- KROK B: Ustalamy wysokosc rabatu na podstawie zmiennej
    IF @PreviousOrdersCount = 0
        SET @NewDiscount = 0.00;
    ELSE IF @PreviousOrdersCount IN (1, 2)
        SET @NewDiscount = 0.05;
    ELSE IF @PreviousOrdersCount = 3
        SET @NewDiscount = 0.10;
    ELSE
        SET @NewDiscount = 0.20;

    -- KROK C: Wykonujemy UPDATE tylko dla tego JEDNEGO konkretnego wiersza
    UPDATE [Order Details]
    SET Discount = @NewDiscount
    WHERE OrderID = @CurrentOrderID AND ProductID = @CurrentProductID;

    -- KROK D: Pobieramy KOLEJNY wiersz z kursora. Jesli wierszy zabraknie,
    -- funkcja @@FETCH_STATUS zwroci -1 i petla sie zakonczy.
    FETCH NEXT FROM order_cursor INTO @CurrentOrderID, @CurrentProductID,
        @CurrentOrderDate;

END

-- 5. CLOSE i DEALLOCATE: Zamykamy i zwalniamy pamiec z kursorem
CLOSE order_cursor;
DEALLOCATE order_cursor;

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;

    -- Zabezpieczenie: jesli byl blad, upewniamy sie, ze kursor nie zawisl w pamieci
    IF CURSOR_STATUS('global', 'order_cursor') >= -1
        BEGIN
            CLOSE order_cursor;
            DEALLOCATE order_cursor;
        END

    THROW;
END CATCH
END;
GO

```

16 Przykładowe Kolokwium 3 - Rozwiązania

script1.sql

```

-- Tworzenie bazy danych edu_courses
CREATE DATABASE edu_courses;
GO

USE edu_courses;
GO

-- Tworzenie tabeli users_user
CREATE TABLE users_user (
    user_id INT IDENTITY(1,1) NOT NULL,
    email NVARCHAR(255) NOT NULL,
    first_name NVARCHAR(200) NOT NULL,
    last_name NVARCHAR(200) NOT NULL,
    is_active BIT NOT NULL,
    age INT,

```

```

    CONSTRAINT PK_users_user PRIMARY KEY (user_id)
);

-- Tworzenie tabeli course
CREATE TABLE course (
    course_id INT IDENTITY(1,1) NOT NULL,
    course_name NVARCHAR(100) NOT NULL,
    base_price MONEY NOT NULL,
    planned_groups_amount INT DEFAULT 1,
    date_start DATE,
    date_end DATE,
    is_active BIT DEFAULT 1,
    CONSTRAINT PK_course PRIMARY KEY (course_id)
);

-- Tworzenie tabeli group (wymaga nawiasow kwadratowych bo to slowo kluczowe)
CREATE TABLE [group] (
    group_id INT IDENTITY(1,1) NOT NULL,
    group_type NVARCHAR(25) DEFAULT 'zajeciowa',
    course_id INT NOT NULL,
    max_group_capacity INT,
    CONSTRAINT PK_group PRIMARY KEY (group_id),
    CONSTRAINT FK_group_course FOREIGN KEY (course_id) REFERENCES course(course_id)
);

-- Tworzenie tabeli group_timetable
CREATE TABLE group_timetable (
    group_id INT NOT NULL,
    room NVARCHAR(10),
    datetime_start DATETIME,
    datetime_end DATETIME,
    CONSTRAINT FK_timetable_group FOREIGN KEY (group_id) REFERENCES [group](group_id)
);

-- Tworzenie tabeli course_enrollment
CREATE TABLE course_enrollment (
    user_id INT NOT NULL,
    group_id INT NOT NULL,
    enrollment_date DATETIME,
    total_cost MONEY,
    discount_type VARCHAR(100) DEFAULT 'bezwarunkowy',
    discount_value MONEY,
    is_completed BIT DEFAULT 0,
    is_dropped BIT DEFAULT 0,
    CONSTRAINT PK_course_enrollment PRIMARY KEY (user_id, group_id),
    CONSTRAINT FK_enrollment_user FOREIGN KEY (user_id) REFERENCES users_user(user_id),
    CONSTRAINT FK_enrollment_group FOREIGN KEY (group_id) REFERENCES [group](group_id)
);

```

script2.sql

```

USE edu_courses;
GO

-- a. Dodanie kolumny phone_number do tabeli users_user
ALTER TABLE users_user
ADD phone_number VARCHAR(25);

-- b. Usuniecie kolumny age z tabeli users_user
ALTER TABLE users_user
DROP COLUMN age;

-- c. Dodanie sprawdzenia, czy date_start jest wcześniejsza niz date_end
ALTER TABLE course
ADD CONSTRAINT CHK_Course_Dates CHECK (date_start < date_end);

```

script3.sql

```

USE edu_courses;
GO

-- 1. Wstawianie danych do tabeli users_user
INSERT INTO users_user (email, first_name, last_name, is_active, phone_number)
VALUES
('j.kowalski@poczta.pl', 'Jan', 'Kowalski', 1, '111-222-333'),
('a.nowak@poczta.pl', 'Anna', 'Nowak', 1, '444-555-666'),
('m.wisniewski@poczta.pl', 'Michal', 'Wisniewski', 0, '777-888-999');

-- 2. Wstawianie danych do tabeli course
INSERT INTO course (course_name, base_price, planned_groups_amount, date_start, date_end, is_active)

```

```

VALUES
('Podstawy SQL', 1500.00, 2, '2024-01-10', '2024-02-10', 1),
('Zaawansowany Python', 2500.00, 1, '2024-03-01', '2024-04-15', 1),
('Analiza Danych', 2000.00, 2, '2024-05-01', '2024-06-30', 1);

-- 3. Wstawianie danych do tabeli group
INSERT INTO [group] (group_type, course_id, max_group_capacity)
VALUES
('wykladowa', 1, 50),
('zajeciowa', 1, 20),
('zajeciowa', 2, 15);

-- 4. Wstawianie danych do tabeli group_timetable
INSERT INTO group_timetable (group_id, room, datetime_start, datetime_end)
VALUES
(1, 'Aula A', '2024-01-12 16:00:00', '2024-01-12 18:00:00'),
(2, 'Lab 101', '2024-01-13 10:00:00', '2024-01-13 12:00:00'),
(3, 'Lab 205', '2024-03-05 18:00:00', '2024-03-05 20:00:00');

-- 5. Wstawianie danych do tabeli course_enrollment
-- Zauwaz, ze uzywamy ID wygenerowanych powyzej (user_id od 1 do 3, group_id od 1 do 3)
INSERT INTO course_enrollment (user_id, group_id, enrollment_date, total_cost, discount_type,
discount_value, is_completed, is_dropped)
VALUES
(1, 2, '2024-01-05 10:30:00', 1400.00, 'bezwarunkowy', 100.00, 0, 0),
(2, 2, '2024-01-08 14:15:00', 1500.00, 'brak', 0.00, 0, 0),
(3, 3, '2024-02-20 09:00:00', 2500.00, 'brak', 0.00, 0, 1);

```

Task2

```

USE edu_courses;
GO

-- 1. Czyste laczenie tabel users_user i course_enrollment.
-- Baza najczesciej laczy te tabele po kolumnie user_id. W tabeli users_user
-- jest to klucz glowny (wiec ma juz indeks), ale w course_enrollment to klucz
-- obcy, wiec warto dodac na nim indeks niezgrupowany, aby bardzo przyspieszyc JOIN.
CREATE NONCLUSTERED INDEX IX_course_enrollment_user_id
ON course_enrollment (user_id);

-- 2. Unikalnosc email w tabeli users_user.
-- Tworzymy indeks unikalny niezgrupowany. Wymusza on, aby w bazie nie
-- zarejestrowaly sie dwie osoby z tym samym mailem, a jednocześnie
-- drastycznie przyspiesza logowanie (wyszukiwanie po mailu).
CREATE UNIQUE NONCLUSTERED INDEX UX_users_user_email
ON users_user (email);

-- 3. Czyste wyszukiwanie danych wedlug daty rozpoczecia i zakonczenia kursu.
-- Tworzymy zlozony (wielokolumnowy) indeks niezgrupowany na tabeli course.
CREATE NONCLUSTERED INDEX IX_course_dates
ON course (date_start, date_end);

-- 4. Zlozony zgrupowany indeks w tabeli course_enrollment.
-- Tabela ta ma klucz glowny (PK) skladajacy sie z (user_id, group_id).
-- W SQL Server Primary Key z definicji staje sie indeksem zgrupowanym (CLUSTERED),
-- wiec technicznie ten wymog zostal spelniony w zadaniu 1.
-- Gdybysmy jednak musieli go utworzyc jawnie i recznie, skladnia wyglada tak:
CREATE CLUSTERED INDEX CX_course_enrollment_user_group
ON course_enrollment (user_id, group_id);

-- 5. Filtracja userow wedlug imienia i nazwiska.
-- Tworzymy zlozony indeks niezgrupowany. Zazwyczaj w aplikacjach filtruje sie
-- "Kowalski Jan", dlatego dobrym zwyczajem jest ustawienie nazwiska jako
-- pierwszej kolumny w kluczu indeksu, a imienia jako drugiej.
CREATE NONCLUSTERED INDEX IX_users_user_names
ON users_user (last_name, first_name);

```

Procedura

```

USE edu_courses;
GO

CREATE OR ALTER PROCEDURE EnrollUserToCourse
    @email NVARCHAR(255),
    @course_id INT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY

```

```

BEGIN TRANSACTION;

-- Zmienne lokalne do przechowania informacji z bazy
DECLARE @user_id INT;
DECLARE @is_user_active BIT;
DECLARE @is_course_active BIT;
DECLARE @base_price MONEY;
DECLARE @available_group_id INT;
DECLARE @user_courses_count INT;
DECLARE @discount_type VARCHAR(100);
DECLARE @discount_value MONEY;
DECLARE @total_cost MONEY;
DECLARE @N INT;

-- 1. Obsługa użytkownika (walidacja i ewentualne tworzenie)
SELECT @user_id = user_id, @is_user_active = is_active
FROM users_user
WHERE email = @email;

IF @user_id IS NULL
BEGIN
    -- Użytkownik nie istnieje, więc go tworzymy (wymagane pola to email, first_name,
    -- last_name i is_active)
    INSERT INTO users_user (email, first_name, last_name, is_active)
    VALUES (@email, 'Nowy', 'Użytkownik', 1);

    -- Pobieramy ID nowo utworzonego użytkownika
    SET @user_id = SCOPE_IDENTITY();
    SET @is_user_active = 1;
END
ELSE IF @is_user_active = 0
BEGIN
    -- Rzucamy błąd jeśli użytkownik jest nieaktywny
    THROW 50001, 'Użytkownik jest nieaktywny i nie może zostać zapisany na kurs.', 1;
END

-- 2. Sprawdzenie statusu kursu
SELECT @is_course_active = is_active, @base_price = base_price
FROM course
WHERE course_id = @course_id;

IF @is_course_active IS NULL OR @is_course_active = 0
BEGIN
    THROW 50002, 'Podany kurs nie istnieje lub jest nieaktywny.', 1;
END

-- 3. Szukanie pierwszej wolnej grupy dla tego kursu
-- Zliczamy aktualnych kursantów w każdej grupie i sprawdzamy pojemność
SELECT TOP 1 @available_group_id = g.group_id
FROM [group] g
LEFT JOIN course_enrollment ce ON g.group_id = ce.group_id
WHERE g.course_id = @course_id
GROUP BY g.group_id, g.max_group_capacity
HAVING COUNT(ce.user_id) < g.max_group_capacity
ORDER BY g.group_id;

IF @available_group_id IS NULL
BEGIN
    THROW 50003, 'Brak wolnych miejsc w grupach dla tego kursu.', 1;
END

-- Zabezpieczenie: sprawdzenie czy użytkownik nie jest już zapisany na ten sam kurs
IF EXISTS (
    SELECT 1 FROM course_enrollment ce
    JOIN [group] g ON ce.group_id = g.group_id
    WHERE ce.user_id = @user_id AND g.course_id = @course_id
)
BEGIN
    THROW 50004, 'Ten użytkownik jest już zapisany na ten kurs.', 1;
END

-- 4. Obliczanie kosztów i rabatów na podstawie historii
-- Liczymy ile kursów dotychczas kupił ten klient (z wyłączeniem tych z których zrezygnował)
SELECT @user_courses_count = COUNT(*)
FROM course_enrollment
WHERE user_id = @user_id AND is_dropped = 0;

-- Obecny kurs będzie kursem N-tym
SET @N = @user_courses_count + 1;

```

```

IF @N = 1
BEGIN
    SET @discount_type = 'Pierwszy kurs - 100 zł';
    SET @discount_value = 100.00;

    -- Zabezpieczenie zeby cena nie spadla ponizej 0 jesli kurs kosztuje np 50 zł
    IF @base_price < 100.00
        SET @total_cost = 0;
    ELSE
        SET @total_cost = @base_price - 100.00;
END
ELSE IF @N = 2
BEGIN
    SET @discount_type = 'Staly rabat - 5%';
    SET @discount_value = @base_price * 0.05;
    SET @total_cost = @base_price - @discount_value;
END
ELSE -- Czyli N >= 3 (lojalnosciowy)
BEGIN
    -- N-ty kurs daje rabat n% dodany do stalego 5%
    SET @discount_type = 'Lojalnosciowy (' + CAST(5 + @N AS VARCHAR) + '%)';
    SET @discount_value = @base_price * ((5.0 + CAST(@N AS FLOAT)) / 100.0);
    SET @total_cost = @base_price - @discount_value;
END

-- 5. Fizyczny zapis na kurs
INSERT INTO course_enrollment (user_id, group_id, enrollment_date, total_cost, discount_type
    , discount_value)
VALUES (@user_id, @available_group_id, GETDATE(), @total_cost, @discount_type,
    @discount_value);

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Wycofanie wszystkich operacji w przypadku wylapania jakiegokolwiek bledu
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    -- Wyrzucenie bledu wyzej, zeby uzytkownik/aplikacja wiedzieli co poszlo nie tak
    THROW;
END CATCH
END;
GO

```